

RentEasy

Plateforme de Location de Logements

Rapport Technique — Architecture & Implémentation

Spring Boot 3.5 • Java 17 • Thymeleaf • MySQL

Table des matières

1	Introduction	2
2	Exigences du Système	2
3	Rôles Utilisateurs	3
4	Architecture Backend — Spring Boot	4
5	Architecture de Sécurité	4
6	Conception de la Base de Données	5
7	Interface Utilisateur — Thymeleaf	6
8	API REST	7
9	Fonctionnalités Implémentées	8
10	Tests & Validation	9
11	Stack Technique	10
12	Conclusion	10

1 Introduction

Contexte du projet

Le marché de la location immobilière connaît une transformation numérique accélérée. Les plateformes de mise en relation entre propriétaires et locataires doivent offrir une expérience fluide, sécurisée et centralisée.

Problématique : Les solutions actuelles souffrent de fragmentation :

- Pas de séparation claire des rôles (propriétaire / locataire / admin)
- Absence de recherche avancée (filtres, pagination)
- Gestion manuelle des réservations et des conflits de dates
- Sécurité insuffisante (pas de JWT, pas de contrôle d'accès)

Objectifs

- **Centre** — CRUD complet pour logements, annonces, réservations
- **Recherche** — Filtres multi-critères (ville, type, prix, disponibilité)
- **Rôles** — Trois profils : Administrateur, Propriétaire, Locataire
- **Sécurité** — Authentification JWT *et* authentification par formulaire (double mécanisme)
- **Internationalisation** — Support Français, Anglais, Arabe

2 Exigences du Système

Exigences Fonctionnelles

Authentification & Gestion

- Inscription / Connexion
- Profil utilisateur
- Gestion des rôles (Admin)
- CRUD utilisateurs (Admin)

Réservations

- Création avec contrôle de conflit
- Confirmation (propriétaire)
- Annulation
- Filtrage par rôle

Logements & Annonces

- CRUD logements
- Upload images / vidéos
- CRUD annonces
- Activation / désactivation

Recherche & Navigation

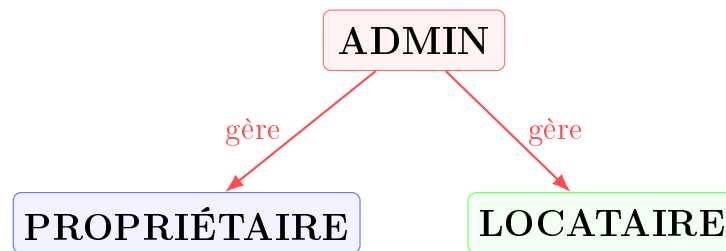
- Recherche multi-critères
- Pagination
- Tableaux de bord par rôle
- Filtrage par disponibilité

Exigences Non-Fonctionnelles

Sécurité	JWT (1h), BCrypt, CSRF partiel, contrôle d'accès RBAC
Performance	Pagination, @EntityGraph, @BatchSize, open-in-view=false
Maintenabilité	Architecture en couches, mappers statiques, DTO séparés
Scalabilité	H2 dev / MySQL prod, profils Spring configurables
Internationalisation	3 bundles (FR/EN/AR), détection par paramètre ?lang=

3 Rôles Utilisateurs

Trois profils → permissions distinctes

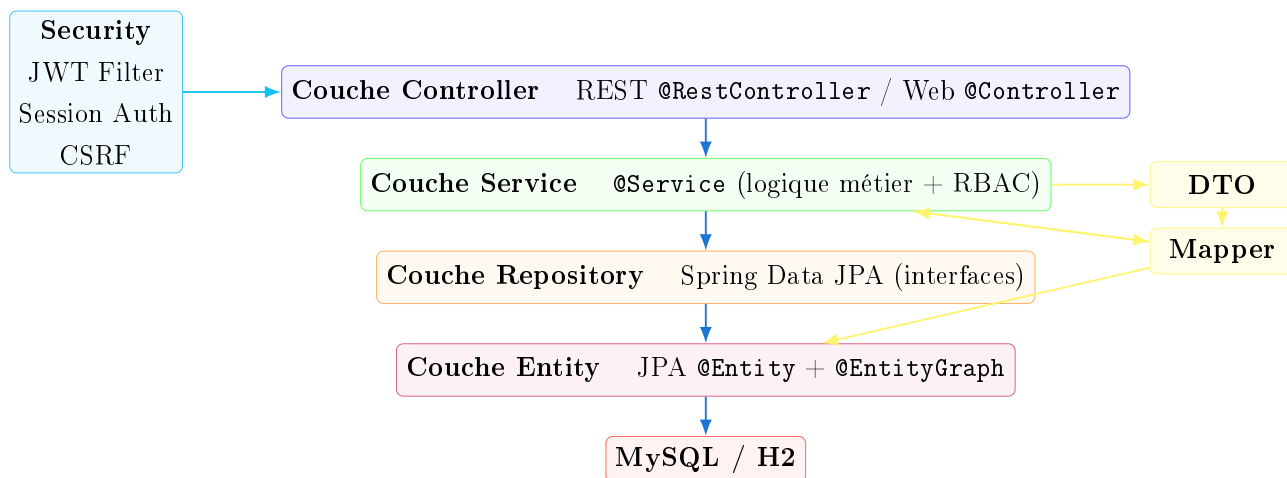


Action	Admin	Propriétaire	Locataire
CRUD Logements	Tous	Ses propres	—
CRUD Annonces	Toutes	Ses propres	—
CRUD Réservations	Toutes	En tant que propriétaire	En tant que locataire
Confirmer réservation	Oui	Ses logements	—
Annuler réservation	Oui	Oui	Ses propres
Dashboard	Global	Ses stats	Ses stats
Gestion utilisateurs	Oui	—	—
Recherche logements	Oui	Oui	Oui

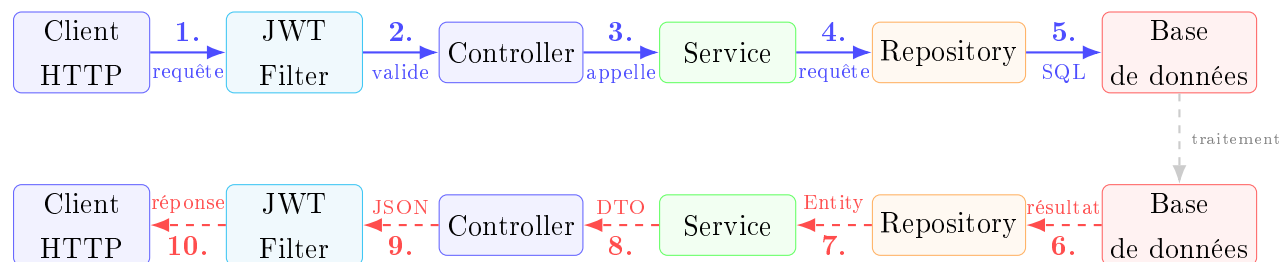
Modèle d'autorisation : Les permissions sont vérifiées au niveau **service** (et non controller). `SecurityUtils.getCurrentUser()` est injecté pour vérifier la propriété des ressources et le rôle.

4 Architecture Backend — Spring Boot

Architecture en couches (Layered Architecture)



Flux d'une requête typique

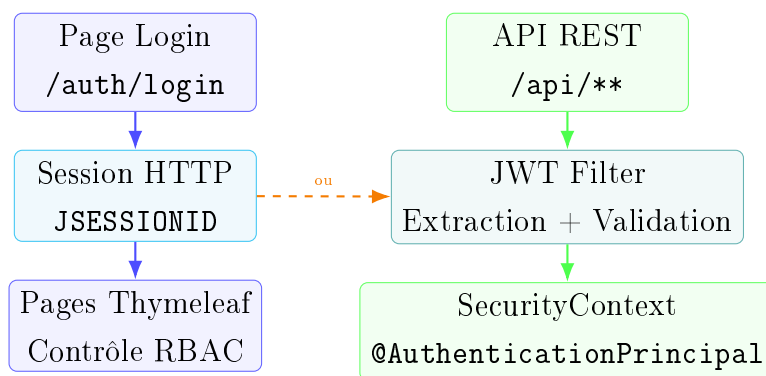


Points clés de l'architecture

- **Double authentification** : Formulaire (session) pour les pages web ; JWT Bearer pour l'API REST
- `SessionCreationPolicy.IF_REQUIRED` — un seul mécanisme actif à la fois
- **CSRF désactivé** pour `/api/**`, **activé** pour les routes web
- `spring.jpa.open-in-view=false` — pas de lazy loading hors `@Transactional`
- Mappers **statiques** (pas MapStruct) — dans le package `mapper/`
- `ApiResponse<T>` enveloppe toutes les réponses REST
- Génération de seed data via `DataInitializer` (`CommandLineRunner`)

5 Architecture de Sécurité

Double mécanisme : Sessions Web + JWT API



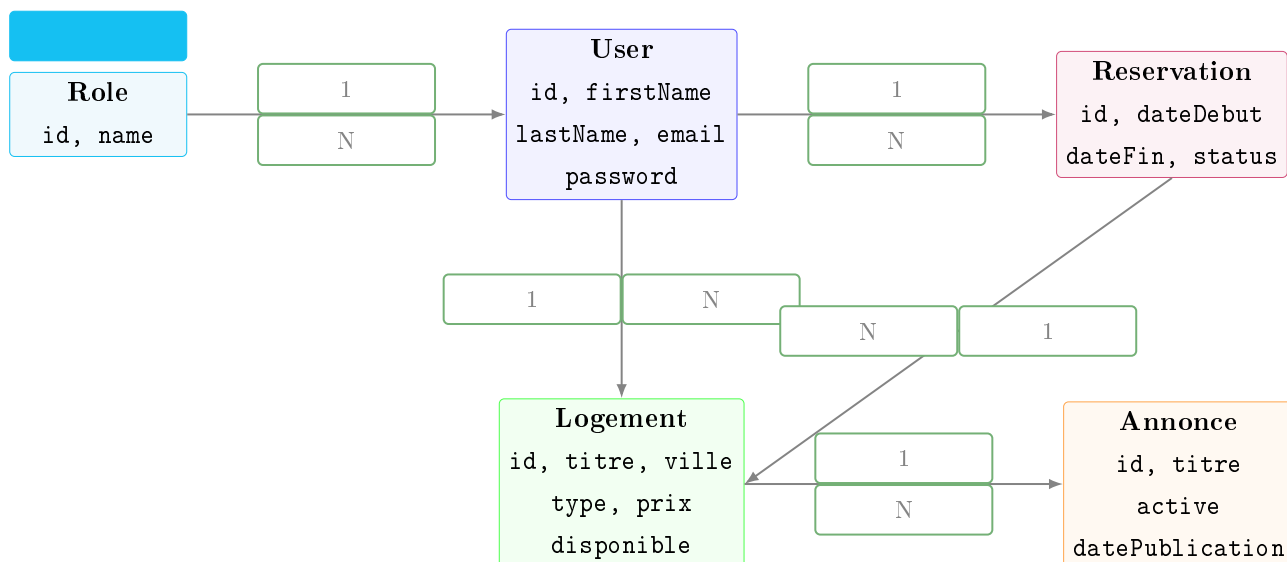
Composant	Rôle
JwtService	Génération/validation token HS256, expiration 1h, clé Base64 256-bit
JwtAuthenticationFilter	OncePerRequestFilter, intercepte Authorization: Bearer, positionné avant UsernamePasswordAuthenticationFilter
CustomUserDetailsService	loadUserByUsername(email) — email comme identifiant
CustomUserDetails	getUsername() retourne email , autorités sans préfixe ROLE_
SecurityUtils	Injection de l'utilisateur courant dans les services
SecurityConfig	formLogin avec usernameParameter("email"), redirection par rôle

Contrôle d'accès — RBAC

- Routes /api/admin/**, /dashboard/admin/** → réservé ADMIN
- Routes /api/proprietaire/** → réservé PROPRIETAIRE
- Routes /api/locataire/** → réservé LOCATAIRE
- Ownership vérifiée dans les services (propriétaire ≠ éditeur → UnauthorizedActionException)
- Gestion des erreurs : GlobalExceptionHandler (@RestControllerAdvice) convertit les exceptions en codes HTTP appropriés

6 Conception de la Base de Données

Modèle Conceptuel (MCD) et Logique (MLD)



Détail des entités

Entité	Table	Index	Associations
Role	roles	—	1 → N User
User	users	email	N → 1 Role; 1 → N Reservation
Logement	logements	ville, type, prix, disponible	N → 1 User; 1 → N Reservation; 1 → N Annonce
Annonce	annonces	active, logement_id	N → 1 Logement
Reservation	reservations	locataire_id, logement_id	N → 1 User; N → 1 Logement

Contraintes importantes

- CascadeType.ALL + orphanRemoval = true sur Logement → Reservation et User → Reservation
- @BatchSize(size = 20) sur les collections pour éviter N+1
- @EntityGraph chargé dans tous les repositories (pas de lazy loading hors transaction)
- ReservationStatus : EN_ATTENTE → CONFIRMEE → ANNULEE
- Contrôle de chevauchement des dates via requête JPQL personnalisée

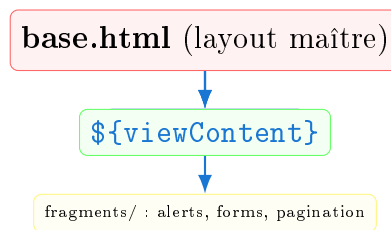
7 Interface Utilisateur — Thymeleaf

Pages web avec rendu côté serveur

Structure des templates

Module	Pages	Description
Authentification	login, register	Standalone (hors layout)
Logements	list, detail, form, mine	CRUD + recherche + upload
Annonces	list, form, mine	CRUD + activation
Réservations	list, detail, form	CRUD + confirmation/annulation
Tableaux de bord	admin, propriétaire, locataire	Statistiques par rôle
Profil	profile	Consultation / modification

Système de layout



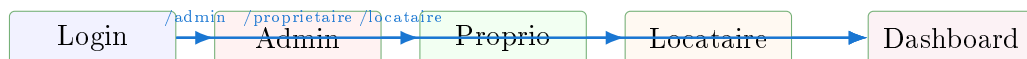
Le système utilise la syntaxe de preprocessing Thymeleaf :

```
~{__${viewContent}__}
```

Les contrôleurs définissent `viewContent = "pages/section/view :: content"`.

Flux de navigation par rôle

Login → Redirection automatique :



Caractéristiques techniques

- Internationalisation : `#{nav.logements}` avec `SessionLocaleResolver`
- CSRF : `~{fragments/forms :: csrf}` dans chaque formulaire POST
- Flash messages : `RedirectAttributes` (clés `success` / `error`)
- Détection de page active : JavaScript (pas de `#request` en Thymeleaf 4)
- Pagination Bootstrap : fragment `pagination.html`
- Upload fichiers : `FileStorageService` → `uploads/{UUID}_{original}`

8 API REST

Endpoints exposés

Méthode	Endpoint	Accès	Description
POST	/api/auth/register	Public	Inscription
POST	/api/auth/login	Public	Connexion → JWT
GET	/api/logements	Auth	Liste paginée
POST	/api/logements	Auth	Création
GET	/api/logements/search	Auth	Recherche filtrée
GET/PUT/DELETE	/api/logements/{id}	Auth	Détail/mod./suppr.
GET/POST	/api/annonces	Auth	Liste / Création
GET	/api/annonces/active	Auth	Annonces actives
GET/PUT/DELETE	/api/annonces/{id}	Auth	Détail/mod./suppr.
GET/POST	/api/reservations	Auth	Liste / Création
GET/DELETE	/api/reservations/{id}	Auth	Détail/suppr.
PUT	/api/reservations/{id}/confirm	Propriétaire	Confirmation
PUT	/api/reservations/{id}/cancel	Auth	Annulation
GET	/api/admin/dashboard	ADMIN	Dashboard global
GET	/api/proprietaire/dashboard	PROPRIETAIRE	Dashboard proprio
GET	/api/locataire/dashboard	LOCATAIRE	Dashboard locataire

Format des réponses

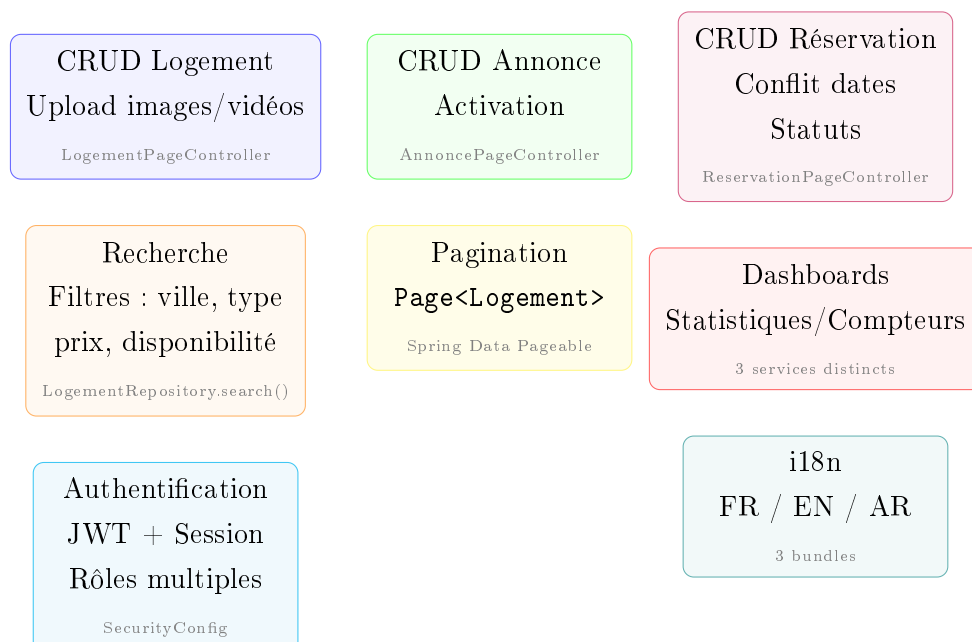
Toutes les réponses utilisent l'enveloppe `ApiResponse<T>` :

```
{ "success": true, "message": "...", "data": {...} }
```

Les erreurs utilisent `ApiErrorResponse` avec `status`, `message`, `timestamp`.

9 Fonctionnalités Implémentées

Modules clés



Détail : Recherche de logements

Endpoint : GET /api/logements/search?ville=&type=&minPrix=&maxPrix=&disponible=
Requête JPQL dynamique (LogementRepository.search()) :

```
SELECT l FROM Logement l WHERE
  (:ville IS NULL OR l.ville LIKE %:ville%) AND
  (:type IS NULL OR l.type = :type) AND
  (:minPrix IS NULL OR l.prix >= :minPrix) AND
  (:maxPrix IS NULL OR l.prix <= :maxPrix) AND
  (:disponible IS NULL OR l.disponible = :disponible)
```

Pagination intégrée via Pageable. EntityGraph {"owner"} pour éviter N+1.

Détail : Réservation — Contrôle de conflit

Avant création, une requête JPQL vérifie l'absence de chevauchement :

```
SELECT COUNT(r) FROM Reservation r
WHERE r.logement.id = :logementId
  AND r.status <> 'ANNULEE'
  AND r.dateDebut < :dateFin
  AND r.dateFin > :dateDebut
```

Si > 0, ReservationConflictException (HTTP 409).

10 Tests & Validation

Bean Validation + Gestion d'erreurs

Validation des entrées

DTO	Annotation	Champ
RegisterRequest	@NotBlank @Email	email
	@NotBlank	firstName, lastName, password
LogementRequestDTO	@NotNull @Positive	prix
	@NotBlank	titre, ville, type
ReservationRequestDTO	@NotNull @Future	dateDebut, dateFin
	@NotNull	logementId
AnnonceRequestDTO	@NotBlank	titre, description
	@NotNull	logementId

Stratégie de gestion des erreurs



`MethodArgumentNotValidException` → retourne une map d'erreurs par champ (HTTP 400).

`RuntimeException` → catch-all HTTP 400.

Pages web : templates `error/403.html`, `error/404.html`, `error/500.html`.

11 Stack Technique

Environnement de développement et production

Couche	Tech
Backend	Spring Boot 3.5, Java 17, Maven
Sécurité	Spring Security, JWT (jjwt 0.11.5, HS256, 1h), BCrypt
Persistance	Spring Data JPA, Hibernate, H2 (dev), MySQL 8 (prod)
Frontend	Thymeleaf, Bootstrap 5.3.3, Bootstrap Icons, Google Fonts
Validation	Bean Validation (Hibernate Validator)
Documentation API	SpringDoc OpenAPI (Swagger UI)
Internationalisation	3 bundles properties (FR/EN/AR)
Fichiers	Upload local vers <code>uploads/</code>
Outils	Lombok, Devtools, Maven wrapper

Profils Spring :

- `dev` → H2 en mémoire, `show-sql=true`, multipart 10MB/20MB
- `prod` (défaut) → MySQL `localhost:3306/renteasy`, `ddl-auto=update`

12 Conclusion

Résumé des réalisations

- Plateforme complète de location avec 3 rôles, authentification duale (JWT + session)
- Architecture Spring Boot en couches respectant les principes SOLID
- API REST complète avec pagination, filtres, et enveloppe de réponse standardisée
- Interface web avec Thymeleaf, internationalisée (FR/EN/AR), responsive

- Sécurité renforcée : RBAC, CSRF, validation des entrées, gestion centralisée des erreurs
- Modèle de données optimisé (index, EntityGraph, BatchSize, cascade)

Améliorations futures

Tests unitaires
et d'intégration

Notifications
email

Paielement en
ligne

CI/CD
Pipeline

Dockerisation
conteneurs

Refresh token
JWT

— Fin du rapport —